

Labor IV (01): Abgabe (30 min, 12 Punkte)

Loesen Sie die untenstehende Aufgabe durch die Implementierung eines ausfuehrbaren Programms in task.cpp und laden die Quelldatei nach TUWEL hoch.

Sie koennen Ihr Programm mittels folgender Zeile kompilieren und ausfuehren:

```
g++ -fmax-errors=1 -std=c++20 task.cpp -o task && ./task
```

Aufgabenstellung

Implementieren Sie jede der folgenden Funktionen als Funktions-Template, sodass die unten vorgegebene Nutzung erfolgreich kompiliert und alle Annahmen (assert) erfuehlt:

1. **transform_if**: Gibt einen neuen Vektor zurueck, in dem jedes Element das die Bedingung erfuehlt durch eine uebergebene Transformationsfunktion veraendert wurde.
2. **any_of**: Gibt *true* zurueck, wenn mindestens ein Element des Vektors die Bedingung erfuehlt, sonst *false*.
3. **min_if**: Gibt das kleinste Element des Vektors zurueck, das die Bedingung erfuehlt.

```
int main() {
    { // usage for type 'int'
        std::vector<int> values = {3, 7, 2, 9, 1, 6};
        auto is_even = [](int v) { return v % 2 == 0; };
        auto triple = [](int v) { return v * 3; };

        assert(any_of<int>(values, is_even) == true);
        assert(min_if<int>(values, is_even) == 2);
        assert(transform_if<int>(values, is_even, triple)
            == std::vector<int>({6, 18}));
    }
    { // usage for type 'double'
        std::vector<double> values = {1.5, 3.0, 0.8, 4.5, 2.0};
        double threshold = 1.5;
        auto above = [threshold](double v) {
            return v > threshold;
        };
        auto halve = [](double v) { return v / 2.0; };

        assert(any_of<double>(values, above) == true);
        assert(std::abs(min_if<double>(values, above) - 2.0) < 1e-5);
        assert(transform_if<double>(values, above, halve)
            == std::vector<double>({1.5, 2.25, 1.0}));
    }
}
```